

TP Évaluation - Déploiement d'une Infrastructure Complète

Niveau : B2-D - Responsable de Projets Informatiques **Type** : Évaluation pratique individuelle **Coefficient** : 40% de la note finale

Objectifs de l'évaluation

Vous devez déployer en production une application web complète (frontend + backend + base de données) sur un serveur Linux, en appliquant les bonnes pratiques DevOps et de sécurité vues durant la formation.

Cette évaluation mesure votre capacité à :

- Configurer un environnement de production de manière autonome
 - Appliquer les concepts DevOps et les bonnes pratiques
 - Documenter vos choix techniques et votre démarche
 - Rechercher et résoudre les problèmes rencontrés
 - Sécuriser une infrastructure web
-

Contexte

Vous disposez de :

- Une application **React** (ou tout autre framework frontend) déjà développée
- Une API **Node.js/Express** (ou tout autre backend) déjà développée
- Un serveur **Ubuntu/Debian** vierge avec accès SSH root
- Les credentials d'accès au serveur

Votre mission : mettre cette application en production de manière professionnelle.

Livrables attendus

1. Documentation technique (40 points)

Créer un fichier **DEPLOYMENT.md** dans votre repository qui documente :

1.1 Architecture déployée (10 points)

- Schéma de l'architecture (diagramme)
- Description des composants (Nginx, Node.js, Base de données, PM2)
- Flux de requêtes (client → serveur)
- Choix techniques et justifications

1.2 Procédure d'installation (15 points)

- Prérequis système
- Étapes d'installation détaillées
- Commandes utilisées avec explications
- Configuration des services
- Variables d'environnement nécessaires

Critère important : Une personne externe doit pouvoir reproduire votre déploiement en suivant votre documentation.

1.3 Guide de maintenance (10 points)

- Procédure de mise à jour de l'application
- Commandes de monitoring
- Localisation des logs
- Procédure de rollback en cas de problème
- Contacts et ressources

1.4 Problèmes rencontrés et résolutions (5 points)

- Liste des difficultés rencontrées
- Solutions apportées
- Sources consultées (documentation, forums, etc.)

2. Infrastructure déployée (40 points)

2.1 Serveur web (10 points)

- Nginx correctement configuré
- Reverse proxy fonctionnel vers l'API
- Serving des fichiers statiques (React build)
- Configuration optimale (gzip, cache, etc.)

2.2 Application backend (10 points)

- API Node.js fonctionnelle
- Gestion par PM2
- Démarrage automatique au boot
- Variables d'environnement configurées
- Logs accessibles

2.3 Base de données (10 points)

- MySQL ou PostgreSQL installé et configuré
- Base de données créée
- Utilisateur avec privilèges appropriés
- Connexion sécurisée
- Sauvegarde automatique configurée

2.4 Accessibilité (10 points)

- Application accessible depuis l'extérieur
- Frontend fonctionnel
- API répondant correctement
- Base de données connectée

3. CI/CD et automatisation (10 points)

- Pipeline GitHub Actions fonctionnelle
- Tests automatiques (si applicable)
- Build automatique
- Déploiement automatique ou semi-automatique
- Documentation du workflow dans **DEPLOYMENT.md**

4. Sécurité (10 points)

- SSH configuré de manière sécurisée
- Firewall (UFW) configuré
- Fail2ban installé et actif
- HTTPS/SSL configuré (Let's Encrypt ou autre)
- Pas de secrets dans le code (variables d'environnement)
- Headers de sécurité Nginx configurés

Critères d'évaluation détaillés

Documentation (40 points)

Critère	Points	Description
Clarté et structure	10	Documentation organisée, bien formatée, facile à suivre
Complétude	10	Tous les aspects couverts, rien d'oublié
Reproductibilité	10	Une personne peut reproduire le déploiement
Qualité technique	10	Justifications pertinentes, bonnes pratiques respectées

Infrastructure (40 points)

Critère	Points	Description
Nginx	8	Configuration correcte, reverse proxy fonctionnel
Node.js/PM2	8	Application stable, logs accessibles, restart automatique
Base de données	8	Correctement configurée, connectée, sécurisée
Fonctionnement global	8	Application complète accessible et fonctionnelle
Optimisations	8	Cache, compression, performances

CI/CD (10 points)

Critère	Points	Description
Pipeline fonctionnelle	5	GitHub Actions configuré et opérationnel
Automatisation	5	Build et/ou déploiement automatique

Sécurité (10 points)

Critère	Points	Description
SSH sécurisé	2	Désactivation password, port non-standard, etc.
Firewall	2	UFW configuré correctement
SSL/HTTPS	3	Certificat valide, redirection HTTP → HTTPS
Bonnes pratiques	3	Secrets protégés, headers sécurité, fail2ban

Déroulement de l'évaluation

Phase 1 : Préparation (15 minutes)

- Lecture du sujet
- Planification de votre approche
- Identification des ressources nécessaires

Phase 2 : Déploiement (2h30)

- Configuration du serveur
- Déploiement de l'application
- Tests et validation

Phase 3 : CI/CD et sécurisation (1h)

- Mise en place de la pipeline
- Configuration de la sécurité
- Optimisations

Phase 4 : Documentation (1h)

- Rédaction du fichier DEPLOYMENT.md
- Capture d'écran / schémas
- Finalisation

Phase 5 : Validation (15 minutes)

- Vérification finale
- Test complet du déploiement
- Soumission des livrables

Ressources autorisées

Vous avez le droit de consulter :

- Vos notes de cours
- La documentation officielle (Nginx, Node.js, PostgreSQL/MySQL, etc.)
- Stack Overflow et forums techniques
- Documentation Notion du cours
- ChatGPT ou autres IA (à mentionner dans votre documentation)

Important : Tout emprunt de code ou de configuration doit être documenté avec sa source.

Consignes techniques

Serveur fourni

- OS : Ubuntu 22.04 LTS ou Debian 11
- RAM : 2GB minimum
- Accès : SSH avec clé privée fournie
- Ports ouverts : 22, 80, 443

Votre application

- Repository Git avec votre code
- Frontend : build de production requis
- Backend : fichier .env.example fourni
- Base de données : script SQL d'initialisation

Contraintes

- L'application doit être accessible via HTTP et HTTPS
 - Les logs doivent être consultables
 - L'application doit redémarrer automatiquement après un reboot du serveur
 - Les secrets ne doivent pas être commités dans Git
-

Scénarios de test obligatoires

Votre déploiement doit passer les tests suivants :

Test 1 : Accessibilité

```
# L'application frontend doit répondre
curl -I http://votre-domaine-ou-ip

# L'API doit répondre
curl http://votre-domaine-ou-ip/api/health
```

Test 2 : Base de données

```
# La base doit être accessible depuis l'application
# Test via un endpoint API qui interroge la BDD
curl http://votre-domaine-ou-ip/api/test-db
```

Test 3 : Persistance

```
# Redémarrer le serveur
sudo reboot

# Attendre 2 minutes, puis vérifier que l'app est revenue
curl http://votre-domaine-ou-ip
```

Test 4 : Logs

```
# Les logs PM2 doivent être accessibles
pm2 logs

# Les logs Nginx doivent exister
sudo tail -n 20 /var/log/nginx/access.log
```

Test 5 : Sécurité

```
# Le firewall doit être actif
sudo ufw status

# SSL doit fonctionner (si configuré)
curl -I https://votre-domaine
```

Barème de pénalités

- Application non accessible : -20 points
- Documentation manquante ou incomplète : -15 points
- Secrets commitées dans Git : -10 points
- SSL non configuré : -3 points
- Firewall non configuré : -3 points
- Pas de démarrage automatique : -5 points
- Logs non accessibles : -3 points
- Retard de soumission : -2 points par 15 minutes

Bonus possibles (+10 points max)

- Monitoring avec dashboard (Grafana, custom, etc.) : +5 points
- Déploiement multi-environnement (staging + prod) : +3 points
- Tests E2E dans la pipeline CI/CD : +3 points
- Script de déploiement automatique complet : +2 points
- Configuration Docker (si non requis) : +3 points
- Sauvegarde automatique de la BDD testée et fonctionnelle : +2 points

Format de soumission

Repository Git

```
votre-repo/
├── frontend/           # Code React
├── backend/            # Code Node.js
├── DEPLOYMENT.md       # Documentation principale
├── .github/
│   └── workflows/
│       └── deploy.yml  # Pipeline CI/CD
├── scripts/
│   ├── deploy.sh       # Script de déploiement (optionnel)
│   └── backup.sh       # Script de backup (optionnel)
├── nginx/
│   └── app.conf         # Configuration Nginx
└── README.md          # Description du projet
```

Fichier DEPLOYMENT.md (structure suggérée)

```
# Documentation de déploiement – [Nom du projet]

## 1. Architecture

[Diagramme et description]

## 2. Prérequis

[Liste des prérequis]

## 3. Installation pas à pas

### 3.1 Préparation du serveur
### 3.2 Installation des dépendances
### 3.3 Configuration de la base de données
### 3.4 Configuration de l'application
### 3.5 Configuration de Nginx
### 3.6 Configuration SSL
### 3.7 Sécurisation

## 4. CI/CD
```

[Description de la pipeline]

5. Maintenance

5.1 Mise à jour de l'application

5.2 Monitoring

5.3 Logs

5.4 Backup et restauration

5.5 Rollback

6. Problèmes rencontrés

[Liste et solutions]

7. Ressources consultées

[Liens et références]

Questions fréquentes

Q : Puis-je utiliser Docker ?

R : Oui, si votre application le nécessite, mais ce n'est pas requis. Documentez votre choix.

Q : Dois-je acheter un nom de domaine ?

R : Non, vous pouvez utiliser l'adresse IP du serveur. Un nom de domaine est un bonus.

Q : Que faire si je bloque ?

R : Consultez la documentation, cherchez sur Internet, mais documentez vos recherches dans DEPLOYMENT.md.

Q : Puis-je utiliser une base de données distante (AWS RDS, etc.) ?

R : Non, la base de données doit être sur le serveur fourni.

Q : Mon application nécessite des services externes (S3, SendGrid, etc.), que faire ?

R : Simulez ces services ou utilisez des alternatives gratuites. Documentez vos choix.

Q : Combien de temps pour obtenir un certificat SSL ?

R : Avec Let's Encrypt : environ 5 minutes si vous avez un nom de domaine.

Conseils

Organisation

1. Commencez par lire entièrement le sujet
2. Planifiez votre approche avant de commencer
3. Testez chaque étape avant de passer à la suivante
4. Documentez au fur et à mesure (pas à la fin)

Priorités

1. **D'abord** : faire fonctionner l'application (frontend + backend + BDD)
2. **Ensuite** : sécuriser (firewall, SSL)
3. **Puis** : CI/CD
4. **Enfin** : optimisations et bonus

Debuggage

- Consultez les logs en cas de problème
- Testez chaque composant individuellement
- Vérifiez les permissions et les chemins
- Validez votre configuration avant de redémarrer les services

Documentation

- Écrivez comme si vous expliquiez à un collègue
- Incluez les commandes exactes utilisées
- Expliquez vos choix techniques
- Documentez les problèmes ET les solutions

Grille de notation finale

Catégorie	Points	Pondération
Documentation technique	40	40%
Infrastructure déployée	40	40%
CI/CD	10	10%
Sécurité	10	10%
Total	100	100%
Bonus (optionnel)	+10 max	-

Critères de réussite

- **Excellent** (90-100 points) : Déploiement complet, sécurisé, documenté de manière professionnelle, avec bonus
- **Très bien** (80-89 points) : Déploiement fonctionnel et sécurisé, documentation complète
- **Bien** (70-79 points) : Déploiement fonctionnel, documentation correcte, quelques manques en sécurité
- **Passable** (60-69 points) : Application déployée mais manques importants (doc, sécu, ou CI/CD)

- **Insuffisant** (< 60 points) : Application non fonctionnelle ou documentation insuffisante
-

Remise des livrables

Deadline

[Date et heure à définir par l'enseignant]

Format de soumission

1. Repository Git accessible (GitHub, GitLab, etc.)
 2. URL de l'application déployée
 3. Credentials SSH pour vérification (si nécessaire)
 4. Email de confirmation avec :
 - Lien vers le repository
 - URL de l'application
 - Toute information nécessaire pour la correction
-